Issue #302 Development Report: Tensor-Native Trace-Aware Local Operator Realization

Nathan V. Roberts (AI-generated)

June 1, 2026

1 Summary

In this issue, we implemented the first tensor-native trace-aware local matrix-free operator-realization path on top of the tensor-skeleton substrate established in #300. The resulting implementation supports a modest but representative mixed volume/trace boundary subset, including both value-trace and flux-trace cases, together with simple explicit parity handling for the flux-like case.

The final implementation also clarified and improved an important semantic point in the local-layout story. During development, it became clear that the original trace-local interpretation in `TensorDofLayout` mixed local and global ideas in a way that was not sustainable. In particular, using the component-global active-vertex count as the distinguished trace-component extent was inconsistent with the otherwise local interpretation of `TensorDofLayout`. The final implementation corrected this: the distinguished trace component now contributes a *local support extent of one*, while the actual skeleton-support location is carried by the tensor skeleton entity used during trace-aware realization.

Thus the final outcome of #302 is somewhat stronger and cleaner than the original plan. We did not merely add a first trace-aware boundary apply path; we also tightened the local-layout semantics enough that the resulting realization model is more coherent for future work.

2 Implemented functionality

The implemented work can be grouped into several main pieces.

2.1 1. Trace/flux semantic helpers

We extended `TensorVar` with internal semantic helper predicates that allow the operator layer to distinguish among:

- trace-like variables,
- value-trace variables, and
- flux-like trace variables.

Publicly, the current `TENSOR_TRACE` umbrella model remains intact, but the realization layer now has a simple and explicit way to branch on the semantics that matter for the supported boundary subset.

2.2 2. Matrix-free factory support for representative boundary terms

Before this issue, the matrix-free factory rejected all boundary terms wholesale. That blanket rejection was relaxed. The factory now accepts representative supported mixed volume/trace boundary terms while continuing to reject unsupported boundary terms conservatively.

At present, the accepted boundary subset is intentionally modest:

- exactly one side must be trace-like,
- both trial and test operators must be value-only in the supported boundary path, and
- unsupported combinations continue to be rejected explicitly.

2.3 3. Lower-dimensional skeleton quadrature and geometry

We added explicit lower-dimensional quadrature and physical-point generation for tensor skeleton entities. For a skeleton entity orthogonal to component d, the implementation now:

- identifies the retained components,
- builds tensor-product quadrature over those retained components only,
- fixes the orthogonal physical coordinate at the selected skeleton vertex, and
- forms the lower-dimensional physical-point and quadrature-weight data needed by `TensorFunction` coefficient evaluation.

This remains intentionally aligned with the current orthogonal 1D-per-component tensor setting and uses the same `CubatureFactory` / Intrepid2-backed line quadrature path used in the volume realization from #299.

2.4 4. Boundary volume-side orthogonal basis evaluation

One subtle but important part of the implementation is that on a skeleton entity, a volume-like variable still contributes a basis factor in the orthogonal direction. Since the boundary integral is lower-dimensional, that orthogonal factor is not quadrature-varying. Instead, it must be evaluated at the fixed endpoint of the incident orthogonal cell.

The final implementation therefore evaluates the orthogonal component of a volume-like variable at the appropriate fixed reference coordinate $\xi = -1$ or $\xi = +1$, depending on whether the skeleton entity corresponds to the lower or upper endpoint of the incident orthogonal cell. This same interpretation was mirrored in the explicit-reference operator tests.

2.5 5. Flux parity support

We implemented the minimal parity/orientation support requested for this issue. In the current supported setting, this is simply a factor of ± 1 determined by whether the selected skeleton vertex is the left or right endpoint of an incident orthogonal-component cell.

This is enough to support the representative flux-trace interpretation needed for the motivating DPG-style use cases, without introducing a more elaborate general orientation framework.

2.6 6. Local trace-layout semantic correction

The most important semantic correction made during stabilization was the change to `TensorDofLayout`.

Originally, the distinguished trace component contributed a support extent based on the component-global active-vertex count. This turned out to be inconsistent with the local interpretation used elsewhere in `TensorDofLayout`. The final implementation changed this so that:

- `TensorDofLayout` is consistently local in scope, and
- the distinguished trace component contributes a local support extent of 1.

This means that the local trace basis tuple no longer attempts to encode a component-global vertex selector. Instead, the tensor skeleton entity identifies the orthogonal support location, and the local trace tuple simply carries one local support slot in that distinguished component.

This is a substantially cleaner interpretation and is the one that the final trace-aware operator path uses.

2.7 7. Explicit validation for current limitations

We also added explicit validation for the current supported subset.

In particular, the trace-aware boundary realization currently rejects:

- derivative-bearing boundary terms outside the supported first subset, and
- retained-component p-nonuniform situations.

The second restriction is especially important. The current flat local tensor coefficient model and `TensorDofLayout` semantics are not yet a general p-nonuniform trace-local realization framework. Rather than silently mis-handle such cases, the operator now rejects them explicitly.

3 Testing and validation

We added focused tests to cover the new trace-aware path and the supporting semantic changes.

3.1 Descriptor/factory coverage

The tensor bilinear-form tests were extended to verify that the matrix-free factory now accepts a supported mixed volume/trace boundary descriptor subset rather than rejecting all boundary terms indiscriminately.

3.2 Local layout coverage

The `TensorDofLayout` and `TensorMesh` tests were updated to reflect the corrected local trace semantics. In particular, the distinguished trace component now has local support extent 1, and the tests verify that this interpretation is used consistently.

3.3 Trace-aware operator coverage

The most important additions were made in `TensorOperatorTests.cpp`. These include:

- a mixed value-trace boundary apply correctness test against an explicit-reference action,

- a mixed flux-trace boundary parity test, again against an explicit-reference action, and
- rejection tests for unsupported derivative-bearing boundary terms and p-nonuniform retained-component situations.

The final tensor-native test run passed.

4 Important design outcome

The most important design outcome of this issue is the clarification that `TensorDofLayout` should be interpreted as a *local* dof layout.

This means:

- volume components contribute local basis cardinalities,
- the distinguished trace component contributes one local support slot, and
- tensor skeleton entities, rather than local layout extents, identify the actual orthogonal trace-support location.

This is the right direction for the current tensor-native stack, and it avoids forcing component-global support indexing into a fundamentally local layout abstraction.

At the same time, the issue also made clear that more work will eventually be needed to handle genuinely p-nonuniform trace-local realization in a more general way. The current implementation therefore rejects that case explicitly rather than pretending to support it.

5 Deferred work

Several natural extensions remain for later issues.

- Broader trace-aware term support, including derivative-bearing trace terms and eventually trace/trace cases.
- A more general p-nonuniform realization story, likely involving either entity-local layouts or an envelope-style coefficient-space interpretation.
- Additional cleanup or abstraction extraction if the trace-aware operator path grows significantly.
- Lower-dimensional factorized kernels and eventual PAMatrix-aware trace realization.

These are all reasonable follow-on directions, but they were intentionally left out of scope for the present issue.

6 Conclusion

Issue #302 successfully established the first tensor-native trace-aware local matrix-free operator substrate on the tensor skeleton support from #300. The implementation supports a representative mixed volume/trace boundary subset, including both value-trace and flux-trace cases with simple parity handling, and it does so using explicit lower-dimensional geometry, basis action, and coefficient evaluation.

Just as importantly, the stabilization process clarified the proper semantics of the local trace layout and corrected `TensorDofLayout` so that it now behaves coherently as a local coefficient-space descriptor. That correction should make future trace-aware tensor-native work easier to reason about and easier to extend safely.